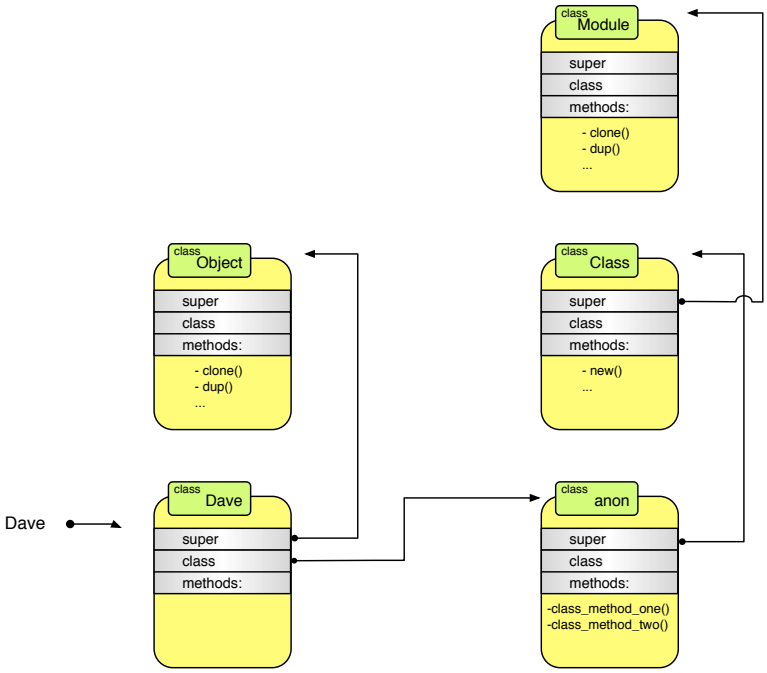


Do you see how the singleton class is created, just as it was for the animal example? The class is inserted as the singleton class of the class Dave, and the original class of Dave is made this new singleton class's parent.

We can now tie together the two uses of self, the current object. We talked about how instance variables are looked up in self, and we talked about how singleton methods defined on self become class methods. Let's use these facts to access instance variables for class objects:

```
class Test
  @var = 99

  def self.var
    @var
  end

  def self.var=(value)
    @var = value
  end
end

puts "Original value = #{Test.var}"
Test.var = "cat"
puts "New value = #{Test.var}"
```

produces:

```
Original value = 99
New value = cat
```

Newcomers to Ruby commonly try to set instance variables inline in the class definition (as we did with @var in the previous code) and then attempting to access these variables from instance methods. As the code illustrates, this won't work, because instance variables are